

# Package guide

How px4\_rta\_mm\_gpr is laid out, how its parts connect, ahead-of-time JAX compilation, the threads, processes and locks, and every import

2026-09-28

## Table of contents

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Start here</b>                                                   | <b>2</b>  |
| <b>2</b> | <b>The repository</b>                                               | <b>2</b>  |
| <b>3</b> | <b>What happens when you run it</b>                                 | <b>3</b>  |
| <b>4</b> | <b>The parts and how they connect</b>                               | <b>4</b>  |
| 4.1      | One control tick (100 Hz, control group) . . . . .                  | 6         |
| 4.2      | One plan (rollout group, and the worker process) . . . . .          | 6         |
| 4.3      | Wind and gains (state and estimation groups) . . . . .              | 6         |
| <b>5</b> | <b>Ahead-of-time JAX compilation</b>                                | <b>7</b>  |
| 5.1      | What JAX does with a function . . . . .                             | 7         |
| 5.2      | Why the node compiles ahead of time . . . . .                       | 7         |
| 5.3      | What is compiled . . . . .                                          | 7         |
| 5.4      | Configuration and the disk cache . . . . .                          | 8         |
| 5.5      | Practical rules . . . . .                                           | 8         |
| <b>6</b> | <b>Concurrency</b>                                                  | <b>9</b>  |
| 6.1      | Threads, the GIL and XLA . . . . .                                  | 9         |
| 6.2      | Executors . . . . .                                                 | 9         |
| 6.3      | Callback groups . . . . .                                           | 9         |
| 6.4      | Where the rollout runs ( <code>--rollout-backend</code> ) . . . . . | 10        |
| 6.5      | Garbage collection . . . . .                                        | 10        |
| 6.6      | Timing statistics . . . . .                                         | 10        |
| <b>7</b> | <b>Shared state and locks</b>                                       | <b>11</b> |
| 7.1      | The pattern: build, then publish with one assignment . . . . .      | 11        |
| 7.2      | Every shared variable . . . . .                                     | 11        |
| 7.3      | The locks there are, and why . . . . .                              | 12        |
| <b>8</b> | <b>The control law, the model and the GP (in brief)</b>             | <b>12</b> |

|           |                                                              |           |
|-----------|--------------------------------------------------------------|-----------|
| <b>9</b>  | <b>Logging, analysis and simulation reuse the same parts</b> | <b>13</b> |
| <b>10</b> | <b>Every import, and what it is for</b>                      | <b>13</b> |
| 10.1      | Standard library . . . . .                                   | 13        |
| 10.2      | ROS 2 . . . . .                                              | 14        |
| 10.3      | Numerics and control . . . . .                               | 15        |
| 10.4      | Data, plots and video . . . . .                              | 16        |
| 10.5      | This package's own modules (who imports what) . . . . .      | 16        |
| <b>11</b> | <b>Where to change what</b>                                  | <b>17</b> |
| <b>12</b> | <b>Glossary</b>                                              | <b>17</b> |

## 1 Start here

This guide explains the package as it is now, top to bottom. It is the one to read first; the other documents go deep on one topic each:

| document             | topic                                                                                                                                                 |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>00 (this one)</b> | layout, data flow, AOT compilation, concurrency and locking, imports                                                                                  |
| 01                   | concurrency in ROS 2 / Python from first principles (executors, GIL, processes, GC)                                                                   |
| 02                   | the multithreaded node: options, benchmarks, flight recorder                                                                                          |
| 03                   | AOT compilation, early-exit rollouts, wind estimation, what caused the crashes                                                                        |
| 04                   | GP feedforward, ground floor, LAND backup, the spatial certificate                                                                                    |
| 06                   | five model fixes (feedback sign, Jacobian domain, row lookup, body-frame velocities, 100 Hz state) and the TV-GPR / GPR × embedding-system comparison |
| 05 (cpp-version)     | the C++ fast loop                                                                                                                                     |

Code references are `path:line` relative to `packages/px4_rta_mm_gpr/` unless stated otherwise.

## 2 The repository

```
PX4_RTA_MM_GPR/
  README.md
  docs/          .qmd sources, PDFs in _output/, figure scripts
  tools/run_sitl.sh  PX4 SITL with this project's parameters
```

```

packages/          colcon --base-paths PX4_RTA_MM_GPR/packages
flight_recorder/   submodule: HDF5 logging (Python + C++ header)
px4_rta_mm_gpr/    the ROS 2 package (ament_python)
package.xml, setup.py, setup.cfg, resource/
launch/           sim_rta_launch.py, only_full_state_relay_launch.py
scripts/
  px4_rta_mm_gpr.py  entry point: options, executor, shutdown
  rta_mm_gpr_node.py OffboardControl: callbacks, shared state
  data_analysis/     notebooks 01-04, figures/, log_files/
px4_rta_mm_gpr/    the library (no ROS, except px4_functions)
  jax_mm_rta/        planar model, TVGPR/GPR, rollout, certificate
  jax_nr/            Newton-Raphson tracker (pitch, yaw rates)
  control_kernels.py AOT: control_step, wind_model, linearize
  concurrency/       RolloutPlan, RolloutEngine, backends, LoopStats
  flight_log/        the node's recorder, the log reader
  px4_functions/     arm, land, heartbeat, setpoints
  utilities/         jax_setup, vehicle constants, yaw unwrapping
  sim/              numerical closed-loop simulation
  analysis.py        summaries, plots, tube videos
test/             pytest: sim + analysis, certificate, lint

```

**Why packages/.** colcon finds a package by its `package.xml`, and it does not look inside another package's directory. With `flight_recorder` (and, on `cpp-version`, two C++ packages) living in this repository, each package needs its own directory next to the others; `packages/` holds them.

**How it builds.** `px4_rta_mm_gpr` is an `ament_python` package: colcon runs `setup.py`, which installs the library (`find_packages` finds `px4_rta_mm_gpr` and `scripts`), `package.xml` and the launch files, and creates two console scripts:

| command                                             | runs                                                                                     |
|-----------------------------------------------------|------------------------------------------------------------------------------------------|
| <code>ros2 run px4_rta_mm_gpr px4_rta_mm_gpr</code> | <code>scripts/px4_rta_mm_gpr.py:main</code> (the node)                                   |
| <code>ros2 run px4_rta_mm_gpr flight_log</code>     | <code>px4_rta_mm_gpr/flight_log/reader.py:main</code><br>(summary / CSV export of a log) |

`flight_recorder` is an `ament_cmake` package (it installs a Python module *and* a C++ header), so colcon `build` builds both from one command. With `--symlink-install` the installed Python files are symlinks to the sources: edits take effect without rebuilding.

**Outside dependencies.** `px4_msgs` (PX4's ROS 2 messages, in `~/ws_sensor_combined`), and pip packages: `jax`, `immrax`, `control`, `scipy`, `h5py`, `pandas`, `matplotlib`, `imageio-ffmpeg`.

### 3 What happens when you run it

`ros2 launch px4_rta_mm_gpr sim_rta_launch.py` runs `px4_rta_mm_gpr --sim --log-file log.log`. Then:

1. `main()` (`scripts/px4_rta_mm_gpr.py`) parses the options into a frozen `RuntimeOptions` data-class and computes the log path, `<workspace>/src/data_analysis/log_files/px4_rta_mm_gpr/`.
2. `rclpy.init()`, then `OffboardControl(sim, options)` (`scripts/rta_mm_gpr_node.py:190`).  
The constructor does all the slow work *before* anything flies:
  - a. vehicle constants (`utilities/sim_utilities.py` or `hardware_utilities.py`), wind EKF, callback groups, publishers and subscriptions;
  - b. `init_jit_compile_nr_rta()`: tunables, GP buffers, `RolloutConfig`, then **compilation**: the rollout worker process is spawned first (it compiles the rollout in parallel), then the control kernels are compiled ahead of time, the first LQR gains computed, and the node waits for the worker’s “ready” (about 4.4 s in total with a warm disk cache);
  - c. the `FlightRecorder` is created with every option as metadata;
  - d. `gc.collect()`; `gc.freeze()` and the generation-2 threshold is raised (see Concurrency);
  - e. the **timers are created last**, so no callback runs before the constructor finishes.
3. The **executor** is built (`--executor multi|single|events`) and `executor.spin()` runs the callbacks. With autosave on, the recorder flushes to disk every 5 s from its own thread.
4. The **mission**, on the node’s clock `t = now() - T0`:

| time               | who flies               | what the node does                                                                                             |
|--------------------|-------------------------|----------------------------------------------------------------------------------------------------------------|
| $t < 15$ s         | PX4 position controller | position setpoint (0, 4, $-12.5$ ) m in SITL; heartbeat; after 10 heartbeats: offboard + arm; warm-up rollouts |
| $15 \leq t < 35$ s | <b>this node</b>        | 100 Hz body-rate + thrust commands from the certified plan; wind EKF and GP learning; replanning               |
| $t \geq 35$ s      | PX4                     | position setpoint above the origin, then land and disarm                                                       |

At any time in the RTA window, if no certified plan exists for more than 20 ms (`--backup-grace`), the node commands PX4 **LAND** and stops sending commands. 5. **Shutdown** (Ctrl-C, or the end of `launch`): `close()` stops the worker process and prints the timing summary; `save_flight_log()` writes the HDF5 file (and the legacy CSV next to it).

## 4 The parts and how they connect

Figure 1 (on the next page) shows the node at a glance.

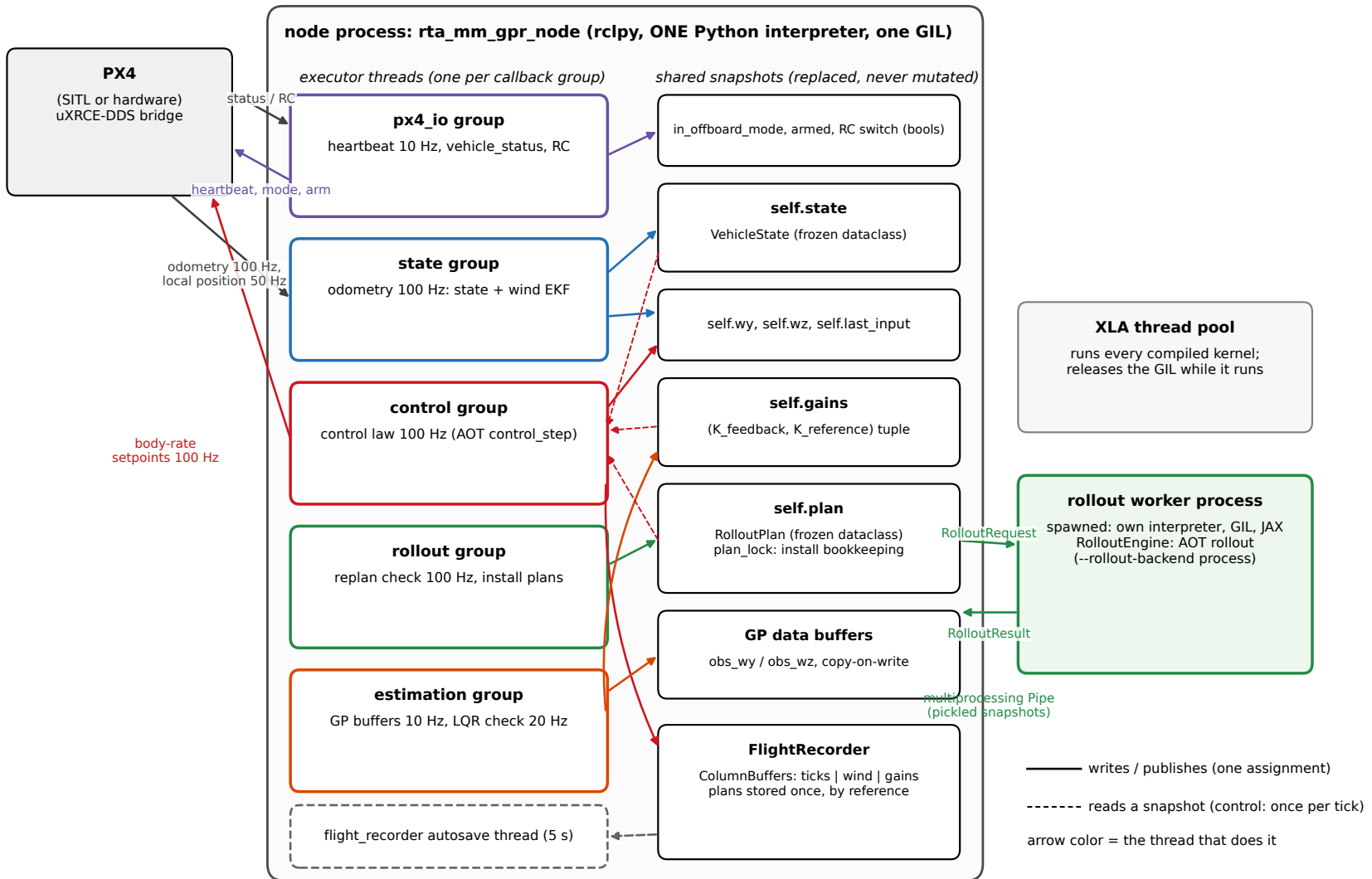


Figure 1: The node process: one executor thread per callback group, the shared snapshots they exchange, the rollout worker process and PX4. Solid arrows: the thread that writes (publishes) a snapshot, in that thread's color. Dashed: the control thread's reads on its hot path. Not drawn, to keep it readable: the rollout thread also reads `self.state`, `self.gains` and the GP buffers when it builds a request; the estimation thread reads `self.state` and the wind estimate.

## 4.1 One control tick (100 Hz, control group)

`control_algorithm_callback` → `control_administrator` (`scripts/rta_mm_gpr_node.py:989`):

1. read `s = self.state` and `plan = self.plan` **once** (they may be replaced at any moment);
2. if `t > plan.collection_time` the plan is no longer certified: count it, and after 20 ms continuously uncertified, `engage_backup()` → PX4 LAND;
3. `traj_idx = plan.index_at(t)`: the plan row for *now*, computed from time, not from a tick counter;
4. **one** compiled call, `kernels.control_step(...)`: the NR tracker (pitch and yaw rates) and the RTA feedback `u = clip(u_ff[i] - K (x - x_ref[i]))` (thrust and roll rate), clipped;
5. thrust → throttle (`get_throttle_command_from_force`), publish the body-rate setpoint;
6. one row into the recorder's ticks buffer.

## 4.2 One plan (rollout group, and the worker process)

`rollout_callback` (:765) runs every 10 ms but only starts a rollout when it is due:

1. `poll()` the backend; a finished result is **installed** (`install_plan`, :843): it becomes an immutable `RolloutPlan`, is published with one assignment `self.plan = plan`, and stored in the log;
2. a rollout is due when `t >= collection_time - lead`, where `collection_time` is when the current plan stops being certified and `lead` is about twice the recent worst rollout latency (**pre-emptive replanning**: the next plan is installed before the current one expires);
3. the request is a **snapshot**: `RolloutRequest(t, state, K_feedback, K_reference, obs_wy, obs_wz, goal)`, all values copied or immutable at that moment;
4. the engine runs the compiled rollout: the reference trajectory under the reference LQR, and the **mixed-monotone embedding** that bounds every state the vehicle can reach, given the GP's wind bounds (mean  $\pm 3$  plus a Lipschitz buffer). It stops at the **first row that fails the certificate** plus a 1 s margin (early exit);
5. the certificate is **spatial**: a row fails when the tube's position bounds ( $p_y$  or  $p_z$ ) are more than 0.25 m + from the reference position, or the tube reaches the 0.3 m floor. , carried by the request, is max(model-mismatch margin 0.05 m, position-estimate uncertainty: 0 in SITL, 3 of EKF2 on hardware); it also widens the initial box (docs 04, 06).

## 4.3 Wind and gains (state and estimation groups)

- **Wind EKF** at the odometry rate (100 Hz), in the state thread, only while this node's commands fly: measured acceleration minus the model's (the compiled `wind_model`) is the wind force estimate (`self.wy`, `self.wz`).
- **GP data**, 10 Hz, estimation thread: one row per buffer (time, input, wind): the y-wind is learned as a function of altitude, the z-wind as a function of lateral position. Each buffer is a 9-row ring, updated **copy-on-write** (see Locks).
- **LQR**, estimation thread: re-linearise the planar model (compiled `linearize`) and solve two LQRs (`control.lqr`) every 1.8 s, or when the yaw error exceeds 5°. The pair of gains is published as one tuple.

## 5 Ahead-of-time JAX compilation

### 5.1 What JAX does with a function

JAX turns a Python function into an XLA program in two steps:

1. **Tracing.** JAX calls the function with *abstract* arguments (only shapes and dtypes) and records every `jnp` operation into a graph (a *jaxpr*). Python control flow runs once, at trace time; loops that must stay loops use `lax.scan`, `lax.while_loop` or `lax.fori_loop`.
2. **Compilation.** XLA optimises the graph (fusing operations, choosing memory layouts) and emits machine code for the CPU. Running it later costs microseconds to milliseconds; compiling costs 0.1–10 s.

`jax.jit(f)` does both lazily, at the first call, and **caches one executable per call signature**: the shapes and dtypes of the arguments, but also details that are easy to miss: a Python `float` is a *weakly typed* scalar while a `float64` array is not; an array already on the device is *committed*. Any difference means a new trace and a new compile.

### 5.2 Why the node compiles ahead of time

The original node “warmed up” by calling each jitted function once in `__init__`. In flight, one argument (`last_input`) became the controller’s own output array: a different signature, so XLA compiled again **in the air** (322 ms: 32 missed control ticks), plus more compiles for slicing and concatenation.

AOT compilation removes the possibility:

```
f64 = lambda *shape: jax.ShapeDtypeStruct(shape, jnp.float64)
compiled = jax.jit(control_step).lower(f64(9), f64(5), f64(4),
    ↪ ...).compile()
```

- `lower(...)` traces for exactly these shapes and dtypes, **without calling** the function;
- `.compile()` returns a `jax.stages.Compiled`: one executable, one signature. **It never recompiles.** Called with anything else it raises `TypeError` immediately, which is what you want to find out on the bench, not in flight;
- every in-flight argument goes through `as_f64(x, shape)` (`control_kernels.py:36`): `np.ascontiguousarray(x, dtype=np.float64).reshape(shape)`. One canonical type, always.

### 5.3 What is compiled

| executable                | built in                           | inputs → outputs                                               | used by                |
|---------------------------|------------------------------------|----------------------------------------------------------------|------------------------|
| <code>control_step</code> | <code>control_kernels.py:54</code> | states, last input, NR reference, plan row, gains → 4 commands | control thread, 100 Hz |
| <code>wind_model</code>   | <code>control_kernels.py:61</code> | NR state, input → model accelerations                          | state thread, 100 Hz   |

| executable | built in                | inputs $\rightarrow$ outputs                                                                                                                                 | used by                                |
|------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| linearize  | control_kernels.py:65   | planar state, input $\rightarrow A, B$<br>( <code>jax.jacfwd</code> )                                                                                        | estimation<br>thread                   |
| rollout    | rollout_backends.py:131 | $t_0$ , state, $x_{\text{pert}}$ , gains, 2 GP buffers,<br>goal, threshold (y, z) $\rightarrow$ tube,<br>reference, feedforward, violation<br>row, rows used | rollout thread<br>or worker<br>process |

`control_step` fuses what used to be about six separate dispatches per tick (NR tracker, slicing, `u_applied`, concatenation, clipping) into one executable: one call from Python per tick.

**Static vs traced arguments.** In the rollout, the things that never change during a flight (horizon, time step, the planar system, its mixed Jacobian, input limits, threshold, floor) are bound with `functools.partial` *before* lowering: they are constants of the compiled program. Only the eight arrays in the table are traced inputs.

**Fixed shapes with early exit.** XLA programs have fixed shapes. The early-exit rollout (`jax_mm_rta/mm_rta.py:307`) uses `lax.while_loop` over preallocated full-horizon arrays filled with NaN and returns `n_valid`, the number of rows it actually computed; the engine trims the result in NumPy. The loop stops at the first failing row plus the margin (3–7 ms instead of 60–190 ms for the full 30 s), and the rows it computes are identical to the full rollout’s (it only depends on earlier rows).

## 5.4 Configuration and the disk cache

`utilities/jax_setup.py` is imported **before anything is compiled** (every module that compiles imports it first):

- `jax_enable_x64 = True`: float64 everywhere (the default is float32);
- `jax_compilation_cache_dir = ~/.cache/px4_rta_mm_gpr/jax`: compiled executables are written to disk and reused by later runs *and by the worker process*. Start-up compilation went from about 20 s to about 4–6 s. `PX4_RTA_JAX_CACHE=''` disables it. Changing the code of a compiled function changes its key, so the cache never serves a stale executable.

## 5.5 Practical rules

- A new argument shape (e.g. more GP rows, a longer horizon) needs a change where the shapes are declared (`_shapes` in `RolloutEngine`, the `lower(...)` calls in `ControlKernels`). It then fails loudly at start-up, never silently in flight.
- Keep Python control flow on *static* values; anything that depends on array values must be `jnp.where`, `lax.cond` or a `lax` loop.
- JAX arrays are immutable: `a.at[i].set(v)` returns a *new* array. (The original GP buffers used `.at[].set()` and discarded the result, so the GP never learned; they are NumPy arrays now.)

## 6 Concurrency

### 6.1 Threads, the GIL and XLA

A Python process runs one interpreter with one **global interpreter lock (GIL)**: only one thread executes Python bytecode at a time. Threads still help here because the heavy work is not Python: a compiled XLA executable **releases the GIL** while it runs, and so do NumPy's large operations and DDS I/O. While the rollout's executable runs on XLA's thread pool, the control thread holds the GIL for its ~0.2 ms tick and continues on schedule. Doc 01 has the full story with timelines.

### 6.2 Executors

An rclpy **executor** waits for ready timers and messages and runs their callbacks. `--executor:`

| option          | what it is                                                                  | callbacks run                                                     |
|-----------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------|
| multi (default) | <code>MultiThreadedExecutor</code> , 5 threads                              | in parallel across callback groups                                |
| single          | <code>SingleThreadedExecutor</code> (the original <code>rclpy.spin</code> ) | one at a time                                                     |
| events          | <code>EventsExecutor</code> ( <code>rclpy.experimental</code> )             | one at a time; the wait/dispatch loop is C++ (much less overhead) |

### 6.3 Callback groups

A **mutually exclusive callback group** never runs two of its callbacks at the same time; different groups may run in parallel on a multithreaded executor. Five groups, so five threads are enough:

| group                         | callbacks (period)                                                                                                  | writes                                                                                                                         |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>px4_io_group</code>     | heartbeat (0.1 s),<br><code>vehicle_status</code> , RC channels                                                     | offboard / armed / RC flags; PX4 mode and arm commands                                                                         |
| <code>state_group</code>      | PX4 <code>vehicle_odometry</code> (100 Hz) + wind EKF;<br><code>vehicle_local_position</code> (50 Hz, acceleration) | <code>self.state</code> , <code>self.wy</code> , <code>self.wz</code> ,<br><code>self.position_sigma</code> , recorder<br>wind |
| <code>control_group</code>    | control law (0.01 s)                                                                                                | <code>self.last_input</code> , recorder ticks,<br>setpoints to PX4                                                             |
| <code>rollout_group</code>    | rollout check (0.01 s)                                                                                              | <code>self.plan</code> ,<br><code>self.collection_time</code> , recorder<br>plans                                              |
| <code>estimation_group</code> | GP buffers (0.1 s), LQR check (0.05 s)                                                                              | GP buffers, <code>self.gains</code> , recorder<br>gains                                                                        |

The rule that makes this work: **each piece of shared state has exactly one writer thread**. Readers in other threads only ever take a snapshot.

## 6.4 Where the rollout runs (`--rollout-backend`)

`concurrency/rollout_backends.py` hides *where* the rollout runs behind `submit(request) / poll()`:

- **thread**: `submit` computes synchronously in the rollout group’s executor thread. With `multi`, that thread is separate from control, and XLA releases the GIL, so control keeps its schedule.
- **process** (recommended, with `--executor events`): a separate OS process with its own interpreter, GIL and JAX runtime. `submit` sends the pickled `RolloutRequest` through a `multiprocessing.Pipe` and returns at once; `poll()` checks the pipe without blocking. Even a single-threaded executor then never waits for a rollout. Details:
  - start method **spawn**, not **fork**: forking a process that already runs JAX/XLA and DDS threads can deadlock the child; **spawn** starts a fresh interpreter that imports only what it needs;
  - the worker is started **first** in `__init__`, so its compilation overlaps with the node’s;
  - `RolloutConfig` (static) is sent once at start; each request/result is a small frozen dataclass of NumPy arrays (cheap to pickle); the result arrays are trimmed to `n_valid` rows;
  - `--rollout-cpus 2,3` pins the worker to CPUs (`os.sched_setaffinity`), away from the control thread;
  - a worker exception is sent back as `('error', traceback)` and raised in the node, never lost.

## 6.5 Garbage collection

CPython frees most objects by reference counting; a separate **cycle collector** runs in generations. A full (generation 2) collection walks every tracked object **while holding the GIL**: measured ~300 ms, i.e. 30 missed ticks, and the JAX/immrax start-up creates hundreds of thousands of long-lived objects.

- `gc.freeze()` after start-up moves all of them to a permanent generation that is never scanned again (about 296 000 objects);
- `--gc-no-full` raises the generation-2 threshold so no full collection runs in flight (young collections still run, ~1 ms); one full collection runs at shutdown;
- a `gc.callbacks` hook measures every in-flight pause and puts it in the log.

## 6.6 Timing statistics

`LoopStats` (`concurrency/loop_stats.py`) wraps each periodic callback in a context manager that records the actual period and execution time into preallocated arrays, under its own small lock. The summary at shutdown (“control: rate 100.0 Hz, period p99 10.3 ms, misses 0”) and the log’s `records/timing/` come from it.

## 7 Shared state and locks

### 7.1 The pattern: build, then publish with one assignment

Most of the node's sharing uses **no lock** at all. A writer builds a complete, immutable object and publishes it by assigning one attribute; a reader reads the attribute **once** and works with that object:

```
# rollout thread (writer)                # control thread (reader)
plan = RolloutPlan(...) # complete      plan = self.plan          # ONE read
self.plan = plan           # publish     row = plan.index_at(t)  # consistent
```

In CPython, assigning an attribute (a reference) is atomic with respect to other threads: a reader sees the old object or the new one, never a mixture. The objects are never modified after publication (frozen dataclasses, tuples; by convention the NumPy arrays inside them are not written either), so the reader needs no lock for as long as it uses them. The same idea in C++ is `std::atomic<std::shared_ptr<const T>>` (the fast loop on cpp-version).

What goes wrong without it: the original node kept `self.x`, `self.y`, `self.vx`, ... as separate attributes updated one by one; another thread could read a new position with an old velocity. The same for a plan whose tube and reference were assigned separately.

### 7.2 Every shared variable

| shared                                                                                        | written by          | read by                                       | mechanism                                                                          |
|-----------------------------------------------------------------------------------------------|---------------------|-----------------------------------------------|------------------------------------------------------------------------------------|
| <code>self.state</code><br>( <code>VehicleState</code> )                                      | state               | control, rollout,<br>estimation               | frozen dataclass, one<br>assignment                                                |
| <code>self.plan</code><br>( <code>RolloutPlan</code> )                                        | rollout             | control, <code>px4_io</code><br>(status line) | frozen dataclass, one<br>assignment; <code>plan_lock</code><br>around installation |
| <code>self.gains</code> (( <code>K_fb</code> ,<br><code>K_ref</code> ))                       | estimation          | control, rollout                              | tuple, one assignment (both<br>gains always from the same<br>linearisation)        |
| GP buffers <code>obs_wy</code> ,<br><code>obs_wz</code>                                       | estimation          | rollout (into the<br>request)                 | copy-on-write: copy, modify<br>the copy, assign                                    |
| <code>self.wy</code> , <code>self.wz</code>                                                   | state               | estimation, control<br>(logged)               | floats, one assignment each                                                        |
| <code>self.position_sigma</code><br>( <code>EKF2</code> , <code>y</code> and <code>z</code> ) | state               | rollout (into the<br>request's )              | a new array, one assignment                                                        |
| <code>self.last_input</code>                                                                  | control             | state (wind model)                            | a new array each tick, one<br>assignment                                           |
| offboard / armed / RC<br>flags                                                                | <code>px4_io</code> | control, heartbeat                            | bools                                                                              |
| warm-up request                                                                               | estimation          | rollout                                       | <code>threading.Event</code>                                                       |

| shared                    | written by                                                                   | read by          | mechanism                                                             |
|---------------------------|------------------------------------------------------------------------------|------------------|-----------------------------------------------------------------------|
| recorder streams          | one thread each<br>(ticks: control,<br>wind: state,<br>gains:<br>estimation) | autosave thread  | <b>ColumnBuffer</b> : row written,<br><i>then</i> the count published |
| recorder records / events | rollout (plans),<br>control (events)                                         | autosave thread  | <b>Recorder._lock</b>                                                 |
| LoopStats samples         | the measured<br>callback                                                     | shutdown summary | <b>LoopStats._lock</b>                                                |

### 7.3 The locks there are, and why

- **plan\_lock** (`threading.Lock`, `rta_mm_gpr_node.py:324`): held while a plan is installed, so the plan number, `collection_time` and the new plan are updated together. The control thread never takes it; it reads `self.plan` once (the plan itself carries its own `collection_time`).
- **Recorder.\_lock** (`flight_recorder`): guards the lists of records and events and the stream registry. Appending a record and the autosave thread’s snapshot of “what is new since the last flush” are short critical sections.
- **Recorder.\_flush\_lock**: one flush at a time (the autosave thread and the final `save()` could otherwise both write the file).
- **Recorder.\_stop** (`threading.Event`): tells the autosave thread to stop; `save()` joins it before the final flush.
- **LoopStats.\_lock**: the measured thread appends while the shutdown code may read.
- **No lock in ColumnBuffer.append**: each stream has one writer; the row is written first and the row count incremented after it, so a concurrent reader (the autosave flush) sees only complete rows. When the buffer grows, it copies into a new array and swaps the reference; a reader holding the old array still has valid rows.

**A known, harmless race.** `self.wy`, `self.wz = wy`, `wz` is two assignments, so a reader could pair a new `wy` with the previous `wz` (25 ms older). Both are slowly varying estimates that only feed the GP buffers and the log; if they ever had to be consistent, they would become one tuple, like the gains.

## 8 The control law, the model and the GP (in brief)

- **Planar model** (`jax_mm_rta/mm_rta.py:75`): state  $(p_y, p_z, h, v, \theta)$  with body-frame velocities  $h, v$ ; inputs thrust  $u_1$  and roll rate  $u_2$ ; wind forces  $w_y, w_z$ .
- **TVGPR** (`jax_mm_rta/TVGPR.py`): time-varying GP regression, squared-exponential kernel in space times a forgetting factor  $(1 - \epsilon)^{|t-t'|/2}$  in time, 9 observations per GP. `GPR.py` is the time-invariant variant, kept as part of the package API.
- **Rollout step** (`_make_step`): the reference moves under the reference LQR (plus the GP mean feedforward on thrust); the tube moves under one of the paper’s **embedding systems** (Appendix A), built from `immrax`’s mixed Jacobians (`irx.mjacM`) of the dynamics: (66)-(67) by default (first order in the feedback, the GP disturbance bounded over the box), or (68)-(69)

/ (64)-(65) with `--embedding`. With TV-GPR they are equally safe and certify the same horizon; (66)-(67) is the cheapest (doc 06).

- **NR tracker** (`jax_nr/`): predicts the output 0.8 s ahead (forward Euler in a `lax.fori_loop`), inverts the Jacobian of that prediction with respect to the input, and adds an integral control-barrier term that keeps the rates within  $\pm 0.8$  rad/s. It flies pitch and yaw; the RTA feedback flies thrust and roll.

## 9 Logging, analysis and simulation reuse the same parts

- **Flight log** (`flight_log/recorder.py` on `flight_recorder`): streams ticks, wind, gains; every plan stored once with the inputs that produced it (`records/plans/<seq>`); timing arrays; events; all options as metadata. Autosaved every 5 s.
- **Numerical simulation** (`sim/planar.py`): the same `RolloutEngine`, `ControlKernels`, `u_applied` and `FlightRecorder`, around an RK4 integration of the planar model and a known wind field (`sim/wind.py`). It writes the same log format, so the analysis works on it unchanged.
- **Analysis** (`analysis.py`): summaries, plots, the `PlanGP` reconstruction of the exact GP a plan used, `tube_violation` (why a certificate ended) and `animate` (the tube videos).

## 10 Every import, and what it is for

### 10.1 Standard library

| import                       | what it is                  | used for                                                                                                                              |
|------------------------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>os</code>              | operating-system interface  | paths, <code>makedirs</code> , environment ( <code>PX4_RTA_JAX_CACHE</code> ), <code>sched_setaffinity</code> (worker CPUs)           |
| <code>sys</code>             | interpreter access          | <code>sys.argv</code> for the command line                                                                                            |
| <code>time</code>            | clocks                      | <code>time.time()</code> (node clock, latencies), <code>perf_counter()</code> (compute times, compile times)                          |
| <code>threading</code>       | threads and synchronisation | <code>Lock</code> ( <code>plan_lock</code> , <code>LoopStats</code> ), <code>Event</code> (warm-up request), recorder autosave thread |
| <code>multiprocessing</code> | processes                   | <code>get_context('spawn')</code> , <code>Process</code> , <code>Pipe</code> for the rollout worker                                   |

| import                                                                     | what it is                               | used for                                                                                                                                                                             |
|----------------------------------------------------------------------------|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>gc</code>                                                            | the cycle garbage collector              | <code>freeze</code> , <code>set_threshold</code> , <code>callbacks</code> (pause measurement), <code>get_objects</code> (heap summary)                                               |
| <code>argparse</code>                                                      | command-line parsing                     | every <code>--option</code> of the node                                                                                                                                              |
| <code>dataclasses</code>                                                   | <code>@dataclass</code>                  | frozen, immutable snapshots: <code>RuntimeOptions</code> , <code>VehicleState</code> , <code>RolloutPlan</code> , <code>RolloutConfig/Request/Result</code> , <code>SimConfig</code> |
| <code>typing</code>                                                        | type hints                               | <code>Optional</code> , <code>Sequence</code> , <code>Dict</code> , <code>Callable</code> , <code>Tuple</code> (documentation and editors)                                           |
| <code>from __future__ import annotations</code>                            | lazy annotations                         | type hints are not evaluated at import time (cheaper, allows forward references)                                                                                                     |
| <code>partial</code> ( <code>functools</code> )                            | bind arguments                           | fix the rollout's static arguments before lowering; <code>@partial(jax.jit, static_argnames=...)</code>                                                                              |
| <code>contextmanager</code> ( <code>contextlib</code> )                    | generator-based <code>with</code> blocks | <code>LoopStats.measure()</code>                                                                                                                                                     |
| <code>inspect</code> , <code>traceback</code>                              | introspection, stack traces              | name the failing function and print the full trace in error handlers; ship worker errors to the node                                                                                 |
| <code>math</code>                                                          | scalar math                              | thrust $\rightarrow$ throttle conversion in the vehicle constants                                                                                                                    |
| <code>glob</code>                                                          | file patterns                            | <code>setup.py</code> (launch files), notebooks (log files)                                                                                                                          |
| <code>textwrap</code><br><code>Counter</code> ( <code>collections</code> ) | text wrapping<br>counting                | reasons in the videos<br>heap summary (object types the GC scans)                                                                                                                    |

## 10.2 ROS 2

| import                                        | what it is                      | used for                                                                              |
|-----------------------------------------------|---------------------------------|---------------------------------------------------------------------------------------|
| <code>rclpy</code>                            | the ROS 2 Python client library | <code>init</code> , <code>ok</code> , <code>shutdown</code>                           |
| <code>Node</code> ( <code>rclpy.node</code> ) | base class of a node            | <code>OffboardControl</code> subclasses it: publishers, subscriptions, timers, logger |

| import                                                                                                                                                                              | what it is                               | used for                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>MutuallyExclusiveCallbackGroup</code><br>( <code>rclpy.callback_groups</code> )                                                                                               | callback groups                          | the five groups above                                                                                                                                                                                                                                                                                                                                             |
| <code>QoSProfile</code> , <code>ReliabilityPolicy</code> ,<br><code>HistoryPolicy</code> , <code>DurabilityPolicy</code><br>( <code>rclpy.qos</code> )                              | quality of service                       | PX4's topics need best-effort,<br>transient-local, depth 1                                                                                                                                                                                                                                                                                                        |
| <code>SingleThreadedExecutor</code> ,<br><code>MultiThreadedExecutor</code><br>( <code>rclpy.executors</code> ); <code>EventsExecutor</code><br>( <code>rclpy.experimental</code> ) | executors                                | <code>--executor</code>                                                                                                                                                                                                                                                                                                                                           |
| <code>px4_msgs.msg</code>                                                                                                                                                           | PX4's uORB<br>messages as ROS 2<br>types | <code>OffboardControlMode</code> ,<br><code>VehicleCommand</code> ,<br><code>TrajectorySetpoint</code> ,<br><code>VehicleRatesSetpoint</code><br>(out); <code>VehicleStatus</code> ,<br><code>RcChannels</code> ,<br><code>VehicleOdometry</code> (state +<br>EKF2 position variance, 100<br>Hz), <code>VehicleLocalPosition</code><br>(acceleration, 50 Hz) (in) |
| <code>launch</code> , <code>Node</code> ( <code>launch_ros.actions</code> )                                                                                                         | ROS 2 launch<br>system                   | the launch files                                                                                                                                                                                                                                                                                                                                                  |
| <code>setuptools</code>                                                                                                                                                             | Python packaging                         | <code>setup.py</code> ( <code>find_packages</code> ,<br>entry points, data files)                                                                                                                                                                                                                                                                                 |

## 10.3 Numerics and control

| import                                                                                                               | what it is                                                                            | used for                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>numpy</code>                                                                                                   | arrays                                                                                | everything outside compiled<br>code: states, buffers, the EKF,<br>logging, canonical <code>float64</code><br>arguments                                                                          |
| <code>jax</code> , <code>jax.numpy</code> as <code>jnp</code>                                                        | NumPy-like arrays<br>+ tracing + XLA<br>compilation +<br>automatic<br>differentiation | the model, GPs, rollout, NR<br>tracker;<br><code>jax.jit(...).lower(...).compile()</code> ,<br><code>ShapeDtypeStruct</code> , <code>jacfwd</code> ,<br><code>vmap</code> , <code>config</code> |
| <code>jax.lax</code> ( <code>scan</code> , <code>while_loop</code> , <code>fori_loop</code> ,<br><code>cond</code> ) | structured control<br>flow that stays<br>inside the compiled<br>program               | the full-horizon rollout, the<br>early-exit rollout, NR's<br>prediction loop, the certificate<br>test                                                                                           |
| <code>jacfwd</code> , <code>jacrev</code> , <code>hessian</code> ( <code>jax</code> )                                | automatic<br>differentiation                                                          | NR's Jacobian of the<br>prediction; <code>linearize</code> for the<br>LQR                                                                                                                       |

| import                                                         | what it is                                               | used for                                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>immrax as irx</code>                                     | interval analysis and mixed-monotone reachability on JAX | <code>irx.interval</code> , <code>icentpert</code> (initial box), <code>i2ut/ut2i</code> (interval embedding vector), <code>mjacM/jacM</code> (mixed Jacobian bounds), <code>ifemb</code> (embedding system), <code>PermutationSystem</code> (base class of the planar model) |
| <code>control</code> (python-control)                          | control-systems toolbox                                  | <code>control.lqr(A, B, Q, R)</code> for both gain matrices                                                                                                                                                                                                                   |
| <code>Rotation</code> ( <code>scipy.spatial.transform</code> ) | rotations                                                | quaternion $\rightarrow$ roll, pitch, yaw in the odometry callback                                                                                                                                                                                                            |

## 10.4 Data, plots and video

| import                                                                                                                                          | what it is                                 | used for                                                                |
|-------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|-------------------------------------------------------------------------|
| <code>flight_recorder: Recorder, ColumnBuffer, git_commit</code>                                                                                | this project's logging library (submodule) | preallocated streams, records, events, HDF5 flushing, autosave          |
| <code>h5py</code> (inside <code>flight_recorder</code> and the reader)                                                                          | HDF5 files                                 | the flight log                                                          |
| <code>pandas</code>                                                                                                                             | data frames                                | log tables ( <code>log.ticks</code> ), summaries                        |
| <code>matplotlib</code> ( <code>pyplot</code> , <code>colors</code> , <code>PolyCollection</code> , <code>patches</code> , <code>lines</code> ) | plotting                                   | figures and video frames (tube boxes are <code>PolyCollections</code> ) |
| <code>imageio_ffmpeg</code>                                                                                                                     | a bundled ffmpeg binary + a frame writer   | MP4 videos ( <code>write_frames</code> )                                |
| <code>PIL</code> (Pillow)                                                                                                                       | images                                     | GIF previews (resize, 96-colour palette)                                |

## 10.5 This package's own modules (who imports what)

| module                              | imports from the package                                                         | imported by                                                                                                           |
|-------------------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>utilities/jax_setup.py</code> | –                                                                                | everything that compiles (imported first)                                                                             |
| <code>jax_mm_rta/</code>            | <code>TVGPR</code> , <code>jax_setup.jit</code>                                  | <code>node</code> , <code>control_kernels</code> , <code>RolloutEngine</code> , <code>sim</code> , <code>tests</code> |
| <code>jax_nr/</code>                | –                                                                                | <code>control_kernels</code> , <code>node</code> (dynamics)                                                           |
| <code>control_kernels.py</code>     | <code>jax_setup</code> , <code>jax_nr</code> , <code>jax_mm_rta.u_applied</code> | <code>node</code> , <code>sim</code>                                                                                  |

| module         | imports from the package                                                  | imported by                          |
|----------------|---------------------------------------------------------------------------|--------------------------------------|
| concurrency/   | jax_mm_rta (inside RolloutEngine, so a spawned worker imports JAX itself) | node, sim                            |
| flight_log/    | flight_recorder                                                           | node, sim, analysis                  |
| px4_functions/ | px4_msgs                                                                  | node only (needs ROS)                |
| utilities/     | –                                                                         | node (adjust_yaw, vehicle constants) |
| sim/           | concurrency, control_kernels, flight_log, jax_mm_rta                      | notebook 04, tests                   |
| analysis.py    | flight_log                                                                | notebooks, tests                     |

from px4\_rta\_mm\_gpr.jax\_mm\_rta import \* and from px4\_rta\_mm\_gpr.px4\_functions import \* in the node are the two star imports left from the original code; `__all__` in each `__init__.py` lists what they bring in.

## 11 Where to change what

| to ...                                     | change                                                                                                                                                                                                         |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| add a command-line option                  | argparse in <code>scripts/px4_rta_mm_gpr.py</code> , a field in <code>RuntimeOptions</code> , use <code>self.options.&lt;name&gt;</code>                                                                       |
| compare wind models / embedding systems    | <code>--gp tv\ static, --embedding uw\ u\ none (node); SimConfig(gp_epsilon=..., embedding=...);</code><br>notebook 05                                                                                         |
| change what the certificate tests          | <code>_row_fails</code> and <code>collection_id_jax</code> ( <code>jax_mm_rta/mm_rta.py</code> ), <code>tube_violation</code> ( <code>analysis.py</code> ), the test in <code>test/test_sim_analysis.py</code> |
| log a new per-tick value                   | <code>TICK_COLUMNS</code> and <code>tick()</code> in <code>flight_log/recorder.py</code> (same order)                                                                                                          |
| add a new periodic job                     | a timer in a callback group; if it writes shared state, publish it as one immutable object                                                                                                                     |
| change an input shape of a compiled kernel | the <code>lower(...)</code> shapes ( <code>ControlKernels</code> , <code>RolloutEngine._shapes</code> ) and the <code>as_f64</code> reshapes                                                                   |
| run without ROS                            | <code>px4_rta_mm_gpr.sim.simulate(SimConfig(...), winds=...)</code>                                                                                                                                            |

## 12 Glossary

**AOT** ahead-of-time compilation: compiling for a declared signature before the first call.

**callback group** a set of callbacks that an executor never runs concurrently (mutually exclusive).

**certificate / certified horizon** the time span over which the reachable tube stays within 0.25 m of the reference position and above the floor; the plan may be flown only within it.

**copy-on-write** modify a copy, then replace the shared reference with it.

**embedding system** a system of twice the dimension whose state is a lower and an upper bound; its trajectory bounds every trajectory of the original system (mixed monotonicity).

**GIL** CPython's global interpreter lock: one thread runs Python bytecode at a time.

**snapshot** an immutable object read once and used for the rest of a callback.

**XLA** the compiler behind JAX; compiled programs run on its own threads and release the GIL.